Reporte

Nombre: Jaime Sebastian Esparza Leyva Alan Joel Sánchez García Jorge Alejandro Jiménez González	**Matrícula**: 2097304 2082464 2097849
Nombre del curso: Componente de lógica de negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: NA	**Actividad**: Preguntas y respuestas
Fecha: 15/10/2026	
Bibliografia (Formato APA) NA	

Preguntas de defensa de la actividad

N.º	Pregunta	Respuesta	Evidencia en el informe
1	¿Qué evidencia del negocio influyó más en la selección del patrón?	Tres evidencias de la Unidad 2: (1) El BRD describe el problema como falta de sincronía entre pedidos, cocina, ventas e inventario: productos que se registran y luego resulta que no hay con qué prepararlos, y diferencias entre ventas e inventario. Eso exige que varios conceptos (producto, receta, ingrediente) se evalúen juntos, no en scripts independientes. (2) El BRD indica que las modificaciones y cancelaciones dependen del estado del pedido: el pedido tiene ciclo de vida y reglas de transición que alguien debe proteger. (3) N-05/N-06 con US-11 y US-12: cada cambio debe dejar usuario, fecha y hora, por lo que la trazabilidad es un invariante. Por eso se eligió Domain Model para lo que cambia estado y Transaction Script solo para consultas de lectura.	Sección 3 (Tablas 10–13); ADR-01
2	¿Qué responsabilidad fue la más difícil de ubicar y cómo tomaron la decisión?	La disponibilidad de un producto. Depende a la vez del Producto, de su receta y de las existencias de varios Ingredientes (RN-02, RN-12). Si la calculaba el Pedido, este tendría que conocer el inventario y se rompería su límite de consistencia; si la calculaba el servicio, la regla quedaría en la Service Layer. Decisión: se creó el Domain Service DisponibilidadProductos. El Pedido solo verifica el resultado (Disponibilidad) y el servicio de aplicación únicamente coordina: carga los objetos, invoca el Domain Service y guarda. Otra responsabilidad difícil fue la autorización para cancelar (SP-05): se dejó fuera del dominio, detrás de la interfaz IAutorizador, porque la autenticación es de infraestructura (SP-09). Es la única regla que no vive en el dominio.	SOLID-02; Sección 5 (tabla «Dónde vive cada regla»); SP-05, SP-09

N.º	Pregunta	Respuesta	Evidencia en el informe
3	¿Qué regla protege el diseño y qué ocurriría si estuviera en una capa incorrecta?	Ejemplo 1: «un pedido solo está completo si todas sus líneas están listas» (RN-06). Vive en el Aggregate Pedido. Si estuviera en la Service Layer o en la interfaz, otro caso de uso o pantalla podría marcar el pedido como Listo sin validarla y la cocina entregaría pedidos incompletos. Ejemplo 2: la disponibilidad (RN-02, RN-12). Está en un Domain Service. Si se copiara en cada servicio, CU-01 y CU-02 podrían aplicarla distinto y volverían las inconsistencias que describe el BRD. En ambos casos el costo de una regla en la capa incorrecta es duplicarla y que dos flujos la apliquen de forma diferente.	RN-02, RN-06, RN-12; Sección 5; Sección 10 (Tabla 45)
4	¿Qué cambió después de la revisión con SOLID?	Se registraron siete hallazgos: seis correcciones y una decisión confirmada. • S: se separó un servicio único de pedidos en cinco servicios (Pedidos, Cocina, Inventario, Catálogo, Promociones); el cálculo de disponibilidad salió del Pedido hacia un Domain Service. • O: las condiciones de promoción pasaron a políticas intercambiables (CondicionPromocion) y las reglas por estado se concentraron en el Value Object EstadoPedido, para no editar cadenas de condicionales. • L (confirmación): se evitó crear PedidoMesa/PedidoDomicilio como subclases; CanalVenta es un Value Object. • I: se separaron los repositorios de escritura de las interfaces de lectura (IConsultas…). • D: toda la infraestructura quedó detrás de interfaces (repositorios, IUnitOfWork, IReloj, IAutorizador).	Sección 8 (Tabla 40, SOLID-01 a SOLID-07)

N.º	Pregunta	Respuesta	Evidencia en el informe
5	¿Qué propuesta de la IA rechazaron o corrigieron y por qué?	Según la bitácora de IA, las correcciones fueron: • Patrones: la recomendación de la IA debía justificarse con las necesidades del caso y no solo con ventajas generales; se mantuvo Domain Model + Transaction Script para consultas con evidencia de la Unidad 2. • Mapa de dominio: algunos límites y reglas no se podían confirmar con la descripción; se conservó lo respaldado y el resto quedó como supuestos (SP-01 a SP-09). • Service Layer/Repository/UoW: una descripción no demuestra que exista código probado; no se afirmó que estuviera terminado. • CQRS y SOLID: no se asumió que hubiera bases de datos separadas ni se declaró cumplimiento de SOLID sin evidencia. • ADR: la IA no puede aceptar una decisión; los cuatro ADR siguen en estado Propuesta. [Completar: el equipo debe agregar aquí ejemplos propios que recuerde haber rechazado.]	Bitácora de IA, iteraciones 1–6; Sección 9
6	¿Qué consecuencia negativa aceptaron al tomar su decisión arquitectural principal?	Dos costos aceptados de forma consciente: (1) Domain Model implica más código y más curva de aprendizaje que Transaction Script, con riesgo de sobrediseño (ADR-01). (2) confirmarPedido modifica el Pedido y varios Ingredientes en una sola transacción, lo que rompe la regla de «un Aggregate por transacción» y puede generar conflictos de versión y reintentos cuando un ingrediente se usa mucho (ADR-03). Se aceptó porque así el inventario esperado se mantiene al día y no se confirma un producto sin existencia, que es el problema central del BRD.	ADR-01, ADR-03; Sección 11 (Tabla 47)

Tabla 3. Preguntas adicionales para la defensa

N.º	Pregunta	Respuesta	Evidencia en el informe
7	¿Por qué no usaron Table Module?	Porque las reglas importantes cruzan conceptos (disponibilidad, consumo de ingredientes) y en Table Module no tendrían dueño: terminarían en un coordinador externo, es decir, un Transaction Script con otro nombre. Su ventaja (reportes por tabla) ya la cubren las consultas.	Tabla 11–12
8	¿Por qué Venta no es un Aggregate?	No tiene invariantes propios: solo resume un pedido cerrado. Se modeló como modelo de lectura que se actualiza con el evento PedidoCerrado (D-04).	Sección 4.8 (D-04)
9	¿Por qué el servicio no contiene las reglas de negocio?	Porque el servicio solo recibe la solicitud, carga objetos, invoca al dominio, pide persistir y devuelve un resultado. Las reglas (RN-xx) están en Pedido, Ingrediente, Producto y los Domain Services. Así se prueban sin base de datos y no se duplican entre casos de uso.	Sección 5
10	¿Qué pasa si falla una operación a la mitad?	Cada método de escritura abre una Unit of Work: si el dominio lanza un error, se revierte todo y no se publican eventos. Los eventos solo se publican después del commit. Un conflicto de versión (concurrencia optimista) también revierte y se puede reintentar.	Sección 6 (Unit of Work); ADR-02, ADR-03
11	¿Por qué CQRS sin bases de datos separadas?	La actividad no lo exige y el volumen de un restaurante no lo justifica. Se separa la responsabilidad (Commands con dominio, Queries con DTO de lectura) sobre la misma base. Costo: mantener DTO y consultas aparte y que VentaResumen dependa del evento PedidoCerrado.	Sección 7; ADR-04
12	¿Qué parte del caso no necesita la separación Command/Query?	Las consultas simples de lectura no cargan Aggregates y se resuelven con Transaction Script. Operaciones sin lógica de negocio que protegen invariantes no justifican un modelo de dominio adicional.	Sección 3 (Tabla 12); Sección 7
13	¿Qué supuestos todavía no están validados?	SP-01 a SP-09, en especial la matriz de estados (SP-02), el umbral crítico por ingrediente (SP-03) y el momento de descuento de ingredientes (SP-04). Tampoco hay historia de usuario para el seguimiento de entrega (SP-07). Los cuatro ADR están en estado Propuesta.	Sección 1 (tabla de supuestos); Sección 11

N.º	Pregunta	Respuesta	Evidencia en el informe
14	¿Qué parte de la arquitectura está ya implementada?	Existe el código de la capa de aplicación (servicios, comandos, consultas y puertos) con 13 pruebas que pasan usando dobles de prueba. El dominio real y la infraestructura aún no están implementados, por lo que la arquitectura completa no se ha probado de punta a punta. [Actualizar si el equipo avanza.]	Carpeta src/aplicacion y tests/
15	¿Cómo usaron y validaron la IA?	La IA propuso borradores (patrones, mapa de dominio, servicios, SOLID, ADR) y el equipo los contrastó con el BRD, el glosario, las historias de usuario y el BPMN, corrigió lo que no tenía respaldo y lo registró en la bitácora. Las decisiones de arquitectura las toma el equipo. Los puntos de validación de consistencia entre diagramas y de uso de IA quedaron pendientes de completar.	Bitácora de IA; Sección 10 (Tabla 46)